Take two long input of integers and implement two versions of divide and conquer algorithm. Check at what value of n one performs better than other.

```
# Given two numbers A and B with n digits each:
# A = aₙ₋₁aₙ₋₂...a₁a₀
# B = bₙ₋₁bₙ₋₂...b₁b₀

# Product P = A × B = Σ (aᵢ × bⱼ × 10^(i+j)) for all i,j in 0 to n-1 implement


def multiply(A:int, B:int)->int:
  str_a=str(A)
  str_b=str(B)

  #get the length of two string
  n=len(str_a)
  m=len(str_b)

  #To check both numbers have same digit
  max_len=max(n,m)
  str_a=str_a.zfill(max_len)
  str_b=str_b.zfill(max_len)

  n=max_len

  p=0#Initialize result as 0

  for i in range(n):
    for j in range(n):
      ai=int(str_a[n-1-i])
      bj=int(str_b[n-1-j])
      p=p+ai*bj*(10**(i+j))
  return p



# For numbers A and B, split each into two halves:
# A = w × 10^m + x
# B = y × 10^m + z

# Original product:
# A × B = wy × 10²ᵐ + (wz + xy) × 10ᵐ + xz

# Karatsuba's optimization:
# Let C = wy
# Let D = xz
# E = (w + x)(y + z) - C - D = wz + xy

# Then:
# A × B = C × 10²ᵐ + E × 10ᵐ + D


def karatsuba(A:int,B:int)->int:
  if A<10 or B<10:
    return A*B

  str_a=str(A)
  str_b=str(B)

  max_len=max(len(str_a),len(str_b))
  str_a=str_a.zfill(max_len)
  str_b=str_b.zfill(max_len)

  m=(max_len+1)//2

  # Split A into w * 10^m + x
  w=int(str_a[:-m]) if m<len(str_a) else 0
  x=int(str_a[-m:])
```

```
    # Split B into y * 10^m + z
    y=int(str_b[:-m]) if m<len(str_b) else 0
    z=int(str_b[-m:])

    #C = wy
    C=karatsuba(w,y)

    #D = xz
    D=karatsuba(x,z)

    #E = (w + x)(y + z) - C - D = wz + xy
    E=karatsuba(w+x,y+z)-C-D

    # A × B = C × 10²ᵐ + E × 10ᵐ + D
    result=C*(10**(2*m))+E*(10**m)+D
    return result



import random
def random_number(n_digits: int)->int:
  if n_digits<=0:
    return 0
  return random.randint(10**(n_digits-1),10**n_digits-1)


import time
def compare():
  digits_length=[10,50,100,200,300,500,1000,1500]
  trails=10
  print("Digit Length  |  Standard Time(S)  | Karatsuba's Time(S)")
  print("-"*60)

  for n in digits_length:
    multiply_times=[]
    karatsuba_times=[]


    for _ in range(trails):
      A=random_number(n)
      B=random_number(n)


      start_time=time.time()
      multiply(A,B)
      multiply_times.append(time.time()-start_time)

      start_time=time.time()
      karatsuba(A,B)
      karatsuba_times.append(time.time()-start_time)

    avg_standard_time=sum(multiply_times)/trails
    avg_karatsuba_time=sum(karatsuba_times)/trails

    print(f"{n:<14}|{avg_standard_time:<20.6f}|{avg_karatsuba_time:<20.6f}")


if __name__ == "__main__":
    compare()
```

```
Digit Length  |  Standard Time(S)  | Karatsuba's Time(S)
------------------------------------------------------------
10            |0.000055            |0.000057
50            |0.003264            |0.001180
100           |0.010012            |0.002574
200           |0.055112            |0.007068
300           |0.131352            |0.012508
500           |0.675439            |0.030303
1000          |5.901726            |0.099607
1500          |25.491235           |0.218136
```

Start coding or generate with AI.